Magpie — Sprint 11 Technical Spec (eBay own-order sync)

Audience: implementing agent (Codex) + Regan. **Builds on** Sprints 0–10 (Phase 4 eBay integration + Sprint 10 `SaleRecord`, live-deployed). **Status: planning / review — implement after approval.**

> Second sprint of Phase 6. Pulls Regan's **own eBay orders** into the sales model with **real fees**, **extending the Phase 4 eBay pattern** (ports + adapters + fakes, Fernet `EbayCredential`, `AuditLog`, `EBAY_ENV`, paste-back consent D-61, the **D-74 HTTP-confinement guard**). **Read-only toward eBay** (no writes/ends/revises). Most of Regan's history is **non-Magpie listings → unmatched**, handled via a **staging queue**. Manual capture (Sprint 10) remains the floor — sync is an enhancement.

---

0. Locked decisions this spec implements

D-96 (Fulfillment + Finances readonly + incremental re-consent), D-97 (one sprint; pre-authorised 11a/11b split if Finances is a rabbit hole), D-98 (staging-queue default; external nullable-item `SaleRecord`s that don't pollute profit), D-99 (idempotency + 90-day default window), D-100 (dedup vs manual; no auto-merge), D-101 (local writes OK; read-only toward eBay), D-102 (honest closure: CI fakes + production validation; matched-SKU real path fake-validated until a genuine Magpie-listed sale).

---

1. What gets built

1. **Scopes + incremental re-consent** — add `sell.fulfillment.readonly` + `sell.finances.readonly` to the credential (paste-back). **Verify the exact current eBay scope strings against official docs before locking the OAuth URL** (D-96).
2. **Adapters + fakes** — `EbayOrderAdapter` (Fulfillment `getOrders`) + `EbayFinancesAdapter` (Finances `getTransactions`) behind ports + `EbayUnavailable`; **fakes** returning canned orders/transactions for CI. HTTP confined to `apps/ebay/` (D-74).
3. **`SaleRecord` model changes** (migration) — make `item` **nullable**; add `is_external`

(bool), `cost_basis_unknown` (bool), and eBay identity fields `ebay_order_id`, `ebay_line_item_id`, `ebay_transaction_id` (+ optional `channel_data` JSON).

4. **Staging model —** `EbayOrderStaging` (raw unmatched order lines awaiting resolution).
5. **Sync engine** — manual-trigger; watermark + 90-day window; matched → `SaleRecord`; unmatched → staging; fees from Finances; idempotent.
6. **Staging-queue + dedup UI** — review unmatched, resolve (link / quick-create / mark external); review duplicate candidates.
7. CI fakes + production validation (D-102).

---

2. `SaleRecord` changes + staging model

`SaleRecord` (extends Sprint 10):

- `item` → **nullable** (`null=True`). Manual records keep an item; **external** sales have none.
- `is_external` (bool, default False) — an accepted non-inventory sale.
- `cost_basis_unknown` (bool, default False) — set when no acquisition cost is known (→ excluded from profit/margin KPIs; revenue/fees still count).
- `ebay_order_id`, `ebay_line_item_id`, `ebay_transaction_id` (nullable; **unique-together (order_id, line_item_id)** among non-superseded sync rows — idempotency key).
- Derived `realised_profit` is **null/omitted when `cost_basis_unknown`** (do not compute a fake profit).

`EbayOrderStaging` (new — raw unmatched lines, not yet a sale):

- eBay `order_id` + `line_item_id` (**unique together** — idempotent staging), `sku` (raw), `quantity`, `line_price`, `sale_date`, `actual_fee` (from Finances when available), `buyer_region` (optional), `raw` (JSON snapshot), `status` (`pending` / `resolved` / `dismissed`), `created_at`.
- Resolution writes a `SaleRecord` and marks the staging row `resolved` (keep it for audit; don't delete).

Backup (`manage.py backup`) includes both tables.

3. Sync flow (manual-trigger)

On "Sync eBay orders":

1. **Window:** from `last_synced` watermark minus a lookback overlap (default **first run = last 90 days**; a separate "backfill full history" action after the 90-day path is proven — D-99). Paginate `getOrders`.
2. **Per order line:**
   - **Idempotency:** if `(order_id, line_item_id)` already exists as a `SaleRecord` or a staging row → skip (no-op). Re-running sync never duplicates.
   - **Fees:** fetch the order's fees via Finances `getTransactions` (SALE transaction for the order) → `actual_fees_total` + breakdown. *(If Finances proves a rabbit hole, D-97 split: ship orders first with Fulfillment's available fee fields, enrich via Finances in 11b.)*
   - **Match by SKU:** `sku` → `InventoryItem`.
     - **Matched** → **dedup check** (D-100): if a likely-duplicate *manual* `SaleRecord` exists (same item, ~same date/price/qty) → create a **duplicate-candidate** flag for the user (don't auto-create, don't auto-merge). Else → create `SaleRecord(provenance=ebay_sync, item, quantity=line qty, sale_price=line total, sale_date, channel=ebay_au, actual_fees_total, + valuation/estimated-fee snapshots)`; item status auto-updates (Sprint 10 lifecycle); **update local listing/draft state** if it maps to a Magpie listing (D-101 — local only, **no eBay write**).
     - **Unmatched** → create an `EbayOrderStaging` row (`pending`).
3. **Update watermark**; **audit** `ebay.order.sync.started/completed` with counts (created / staged / duplicate-flagged / skipped), secret-free.

---

4. Staging-queue + dedup UI

- **Unmatched import queue:** lists `pending` staging rows (SKU, qty, price, date, fee). Per row, the user: **(1) link to an existing item** (→ `SaleRecord` on it), **(2) quick-create an item** (minimal item, then the sale), or **(3) mark external** (→ nullable-item `SaleRecord`, `is_external=True`; prompt for cost basis — if none, `cost_basis_unknown=True`). Resolving marks the staging row `resolved`.

- **Duplicate candidates:** lists matched-order duplicate flags; the user **merges/links/dismisses** (never automatic).
- Both are review surfaces — nothing becomes an accepted sale without an explicit user action (except clean matched orders with no duplicate, which auto-create).

---

5. Re-consent (D-96)

Detect insufficient scope on a sync call → show a clear **re-consent** prompt → request the expanded **read-only** scopes via paste-back (D-61) → update the encrypted credential's stored scopes → **audit** the re-consent (secret-free). **Incremental** (add scopes to the existing credential); only fall back to a full disconnect/reconnect if technically unavoidable.

---

6. Acceptance criteria (D-102 — honest closure)

CI (fakes, dual-lane Validation green) must prove:

1. **Matched path:** a fake order whose SKU matches an item → `SaleRecord(provenance=ebay_sync)` with correct qty/price/date/fees + snapshots; item status updates; local listing state updates.
2. **Unmatched path:** a fake order with an unknown SKU → `EbayOrderStaging(pending)`; resolving via link / quick-create / mark-external produces the right `SaleRecord` (external → nullable item, `is_external`, `cost_basis_unknown` when no cost).
3. **Partial-quantity:** an order line qty 3 against a qty-10 item → remaining 7, `partially_sold` (Sprint 10 lifecycle via sync).
4. **Idempotency:** re-running the same fake sync creates **no duplicates** (SaleRecords or staging) — assert counts stable.
5. **Fee mapping:** Finances fake → `actual_fees_total` + breakdown populated.
6. **Dedup:** a fake matched order resembling an existing manual `SaleRecord` → a **duplicate-candidate flag**, not an auto-created/merged row.
7. **External-sales-don't-pollute:** an external `cost_basis_unknown` sale contributes to revenue/fees but **not** to profit/margin/valuation-accuracy aggregates (assert it's excluded) — *(the aggregate consumers land in Sprint 12; here, assert the flags +*

that `realised_profit` *is null for cost-basis-unknown)*.

8. **Confinement + audit:** HTTP confined to `apps/ebay/` ; sync audited with counts; **no secrets** in audit/logs.

Production validation (Regan present) must prove: OAuth **re-consent** with the expanded scopes; a real `getOrders` + `getTransactions` pull succeeds; his real **unmatched history imports into the staging queue** correctly; audit is secret-free. **The matched Magpie-SKU real path stays fake-validated** until a genuine Magpie-listed item sells — **do not engineer a fake sale to close** (D-102). Fault attribution per D-73 (payload/auth/mapping/scope errors = ours).

`SPRINT11_SYNC_EVIDENCE.md` records both the CI proofs and the production-validation run (secret-free).

---

7. Out of scope (Sprint 11)

Any **write** to eBay (ending/revising/relisting — local state only) · background/scheduled auto-polling (manual-trigger only) · refunds/returns/cancellations reconciliation (v1 imports sales; flag returns as a known gap) · multi-marketplace (eBay AU only) · dashboards/analytics (Sprint 12) · payout/bank reconciliation beyond per-sale fees · multi-currency · FIFO/LIFO. **If you're writing to eBay or building charts, you've left the sprint.**

Hand-off notes

- **Verify the exact eBay scope strings against current docs before locking the OAuth URL** — scope names + the consent flow are the kind of thing that drifts (F-11).
- **Idempotency is load-bearing** (the Sprint-7 duplicate-publish lesson): key everything to eBay `order_id` + `line_item_id` ; re-sync must be a no-op. Test it explicitly.
- **Never fabricate profit:** external/cost-basis-unknown sales count toward revenue + fees only; `realised_profit` stays null until a cost is supplied. This protects the Sprint 12 dashboards from false numbers.
- **Staging is the default for unmatched** — nothing becomes an accepted sale without an explicit user action (except clean matched orders).

- **Read-only toward eBay**: update Magpie's local listing/item state, but make **no eBay write calls** this sprint.
- Re-consent is **incremental** (add scopes), not a destructive reconnect.
- Honest closure: don't engineer a fake matched sale; the real matched path validates when a Magpie-listed item naturally sells.
- When ambiguous, prefer the smaller change that satisfies §6.